Core Engine

Squares

Ownership & Econom y

Com m and Layer

Persistence

P resen tatio n

Random ness

IGameContext

+void sendPlayerToTims(Player &player)
+void triggerAuction(Property &property, std::vector<Player *> bidders)
+void promptPurchase(Player &player, Property &property)
+DiceResult rollForGym()
+void handleDebt(Player &debtor, int amount, Player *creditor)
+int activeCupCount() const
+void awardCup(Player &player)
+TurnPhase getCurrentPhase() const
+std::vector<Player *> getActivePlayers() const
+Player &currentPlayer()
+void nextPlayer()
+Player *findPlayer(std::string const &name) const

WatopolyGame

+int run(int argc, char **argv)
+void start()
+void setupNewGame()
+void setupLoadedGame(std::string const &file)
+void processTurn()
+bool isGameOver() const
+void announceW inner() const
+void redrawBoard() const

Board

+Square &getSquare(int position)
+Square const &getSquare(int position) const
+int positionOf(std::string const &squareName) const
+void attach(DisplayObserver *obs)
+void detach(DisplayObserver *obs)
+void notify(std::vector<Player> const &players)

Player

+std::string getNam e() const
+char getToken() const
+int getCash() const
+int getPosition() const
+void addCash(int amount)
+bool deductCash(int amount)
+void m oveBy(int steps)
+void m oveTo(int position)
+void addProperty(Property *prop)
+void removeProperty(Property *prop)
+std::vector<Property *> const &properties() const
+int getTotalW orth() const
+bool isInTimsLine() const
+void enterTimsLine()
+void leaveTimsLine()
+int tim sTurns() const
+void incrementTimsTurns()
+int timsCups() const
+void addTimsCup()
+bool useTimsCup()
+void declareBankruptcy()
+bool isBankrupt() const
+DiceResult getLastRoll() const

Dice

+DiceResult roll()
+DiceResult roll(int die1, int die2)
+void resetConsecutiveDoubles()
+int consecutiveDoubles() const

DiceResult

+ in t d ie 1
+ in t d ie 2
+ in t s u m
+bool isDoubles

TurnContext

+int currentPlayerIndex
+TurnPhase phase
+int consecutiveDoubles
+DiceResult lastRoll
+Player *debtCreditor

Square

+std::string nam e() const
+int index() const
+void landOn(Player &player, IGameContext &game)

Property

+int purchaseCost() const
+Player *owner() const
+void setOwner(Player *player)
+bool isMortgaged() const
+void m ortgage()
+void unm ortgage()
+int calculateFee(Player &visitor, IGameContext &game)
+void landOn(Player &player, IGameContext &game)

AcademicBuilding

+std::string monopolyName() const
+int im provem entCost() const
+int im provem ents() const
+bool canBuyImprovement(Player const &owner) const
+bool buyIm provem ent()
+bool sellIm provem ent()
+int calculateFee(Player &visitor, IGameContext &game)

Residence

+int calculateFee(Player &visitor, IGameContext &game)

G ym

+int calculateFee(Player &visitor, IGameContext &game)

CollectOSAP

+void landOn(Player &player, IGameContext &game)

DCTimsLine

+void landOn(Player &player, IGameContext &game)

GoToTims

+void landOn(Player &player, IGameContext &game)

GooseNesting

+void landOn(Player &player, IGameContext &game)

TuitionSquare

+void landOn(Player &player, IGameContext &game)
- int tenPercentOfWorth(Player const &player) const

CoopFee

+void landOn(Player &player, IGameContext &game)

SLCSquare

+void landOn(Player &player, IGameContext &game)

NeedlesHallSquare

+void landOn(Player &player, IGameContext &game)

MonopolyBlock

+std::string nam e() const
+bool isOwnedBy(Player const &player) const
+bool hasAnyImprovements() const
+std::vector<Academ icBuilding *> buildings() const

Auction

+void run(Property &property, std::vector<Player *> bidders)
+int currentBid() const
+Player *leadingBidder() const

Trade

+bool validate() const
+void presentTo(Player &target) const
+bool execute()

CommandInterpreter

+void registerDefaultCommands()
+bool executeLine(std::string const &line, IGameContext &game)
+void printInvalidCommand(std::string const &line) const

Command

+std::string nam e() const
+bool execute(IGameContext &game, std::vector<std::string> const &args)
+bool isValidInPhase(TurnPhase phase) const

RollCommand NextCommand TradeCommand ImproveCommand MortgageCommand UnmortgageCommand BankruptCommand AssetsCommand AllCommand SaveCommand

SaveManager

+void save(W atopolyGame const &game, std::string const &filename)
+void load(W atopolyGame &game, std::string const &filename)
+bool validateLoadedState(WatopolyGame const &game) const

DisplayObserver

+void update(Board const &board, std::vector<Player> const &players)

BoardDisplay

+void update(Board const &board, std::vector<Player> const &players)
+std::string renderSquare(Square const &sq) const
+void printAssets(Player const &player) const

RandomEventGenerator

+void seed(unsigned value)
+int rollD ie()
+SLCEvent generateSLCEvent()
+int generateNeedlesHallDelta()
+bool rollUpTheRimCheck(int activeCupCount)

SLCEvent

Back3
Back2
B ack1
Forward1
Forward2
Forward3
GoToTims
AdvanceToOSAP

TurnPhase

PreRoll
Rolling
PostRoll
DebtResolution
TuitionDecision

PieceType

G
B
D
P
S
Dollar
L
T

Dependency Inversion: all Squares and Commands
depend on this narrow interface, not on the
concrete WatopolyGame class.
WatopolyGame implements IGameContext.

Centralises all per-turn mutable state:
- consecutiveDoubles (for triple-doubles jail)
- lastRoll (for forced Tims Line third-turn exit)
- debtCreditor (null means owed to Bank)
- currentPlayerIndex and TurnPhase

Board is a pure container + Observer Subject.
All 40 squares and 10 MonopolyBlocks are
constructed by the free function
buildDefaultBoard(Board &) — Board itself
has no knowledge of specific square types.

Mortgage state:
- Mortgaged properties charge no fee
- Unmortgage cost = 60% of purchase cost
- Properties received via bankruptcy transfer
incur an immediate 10% Bank fee; an extra
10% applies if the recipient unmortgages later

Save/load validates:
- player count (2..6)
- Tims-line encoding
- owner names / BANK reserved name
- improvement bounds / mortgage marker
- total cup count <= 4

Trade is transient: created stack-locally in
TradeCommand::execute and destroyed after
resolution. WatopolyGame does not own or
accumulate Trade objects.

char toChar(PieceType p) maps Dollar -> '$';
all others map to their enum name's first char.
PieceType fromChar(char c) is the inverse.

1

1

1

2..6

1

1

1

1

1 1

1

1

1

1

1

1

1

4 0

1

1 0

1

*

owns

1

*

owner

0..1

1

1

1 1

2..3

1

*